Quiz on Gradescope (10:30-10:40)

Quiz on Gradescope (10:30-10:40)

Select all of the true statements:

a) If your code always runs on 1 core, spinlocks are not a helpful tool to synchronize threads.
b) It is impossible to implement a blocking, one-shot barrier using a Mutex as your only interface to synchronization / the scheduler.
c) Atomic assembly instructions are required to implement multicore synchronization.
d) Calling yield with a InterruptSpinLock held is just a performance bug in multicore, multithreaded executions.
e) Data races are the only source of race conditions.

What is Rust?

Administrivia

- HW3 Progress?
- Class Vibe Change
- Anonymous questions on Ed

Learning Barrier!

- 1 question on Synchronization Primitives
  - Ideas: ISLs vs Mutex, priority inversion, Semaphores, Mesa vs Hoare Monitors, …

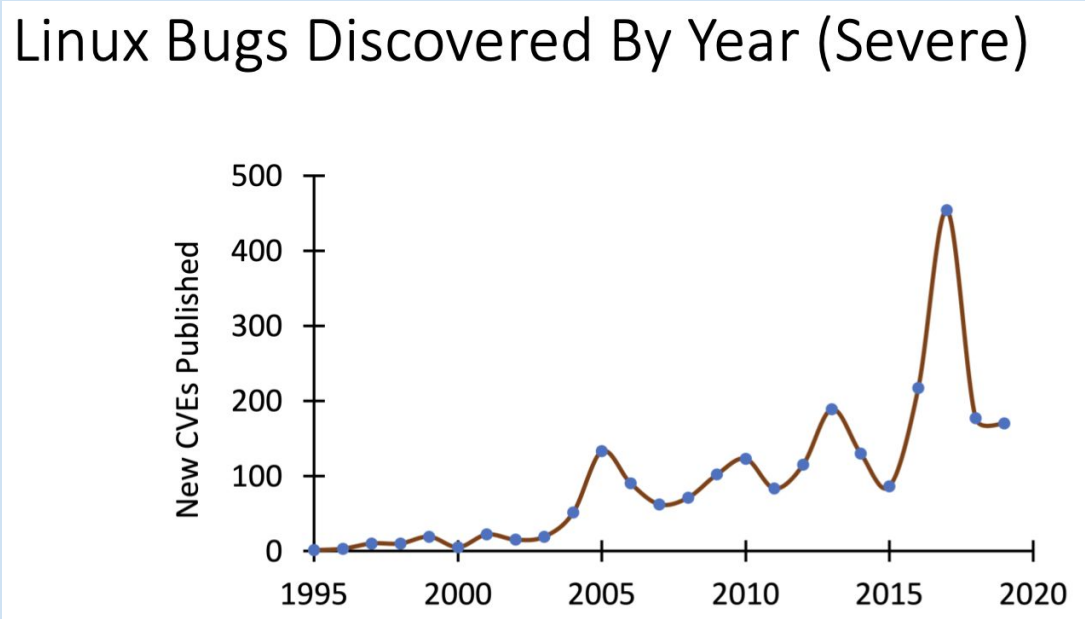
What is Rust?

C is Good

- Direct program control over underlying data layout (unlike Java)
- Direct mapping of high level constructs to underlying machine instructions
  - Predictable performance
- Explicit memory allocation/deallocation
- Linux: 40 million lines of C
- Android: 12 million lines of C

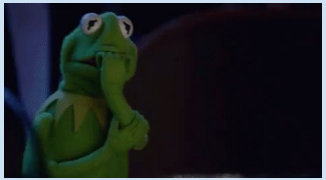
C is Bad

- Direct program control over underlying data layout (unlike Java)
- There are no “high level” constructs
- Explicit memory allocation/deallocation



C is Bad

- What if we forget to initialize data?
  - Undefined behavior
  - Will be set to whatever was in the heap/stack before!
- What if we return a pointer to data on the stack?
  - Undefined behavior
- What if we use a pointer after we free it?
  - Undefined behavior
- What if we free a pointer twice?
  - ahhhhhh



C++ is also bad

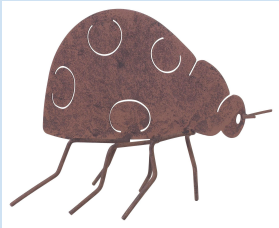
C++

```cpp
#include <vector>
#include <iostream>
int main() {
    std::vector<int> v{5, 10,};
    int &x = v[1];
    v.push_back(20);
    std::cout << x << '\n';   // segfault, or random number
    std::cout << &v[1] << ' ' << &x << '\n';
}
```

Concurrency is Exceptionally Difficult…

Concurrency bugs…

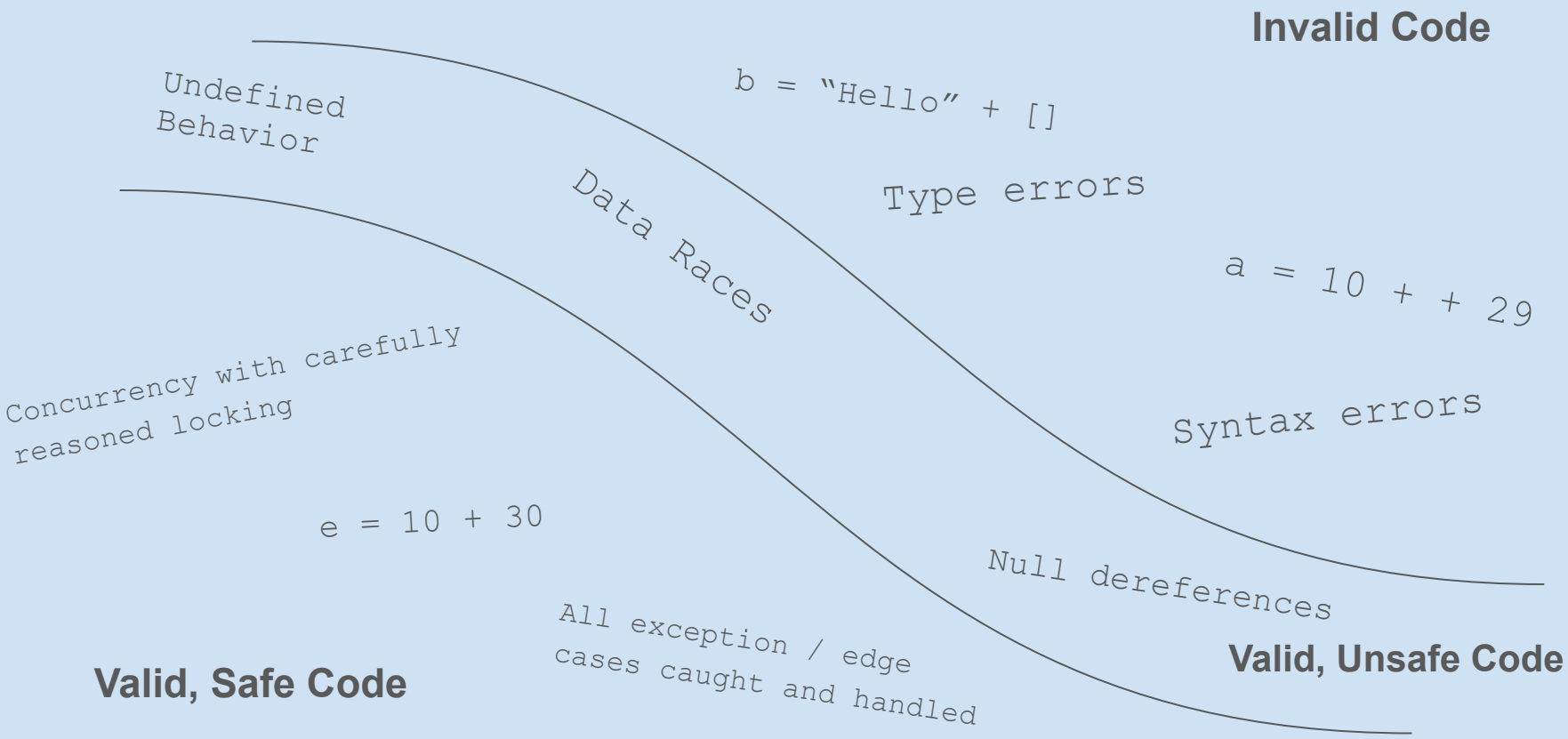
- are often runtime errors, and require extensive test suites to catch
- are hard to avoid, with concurrency being important for performance
- C does nothing to protect you against data races



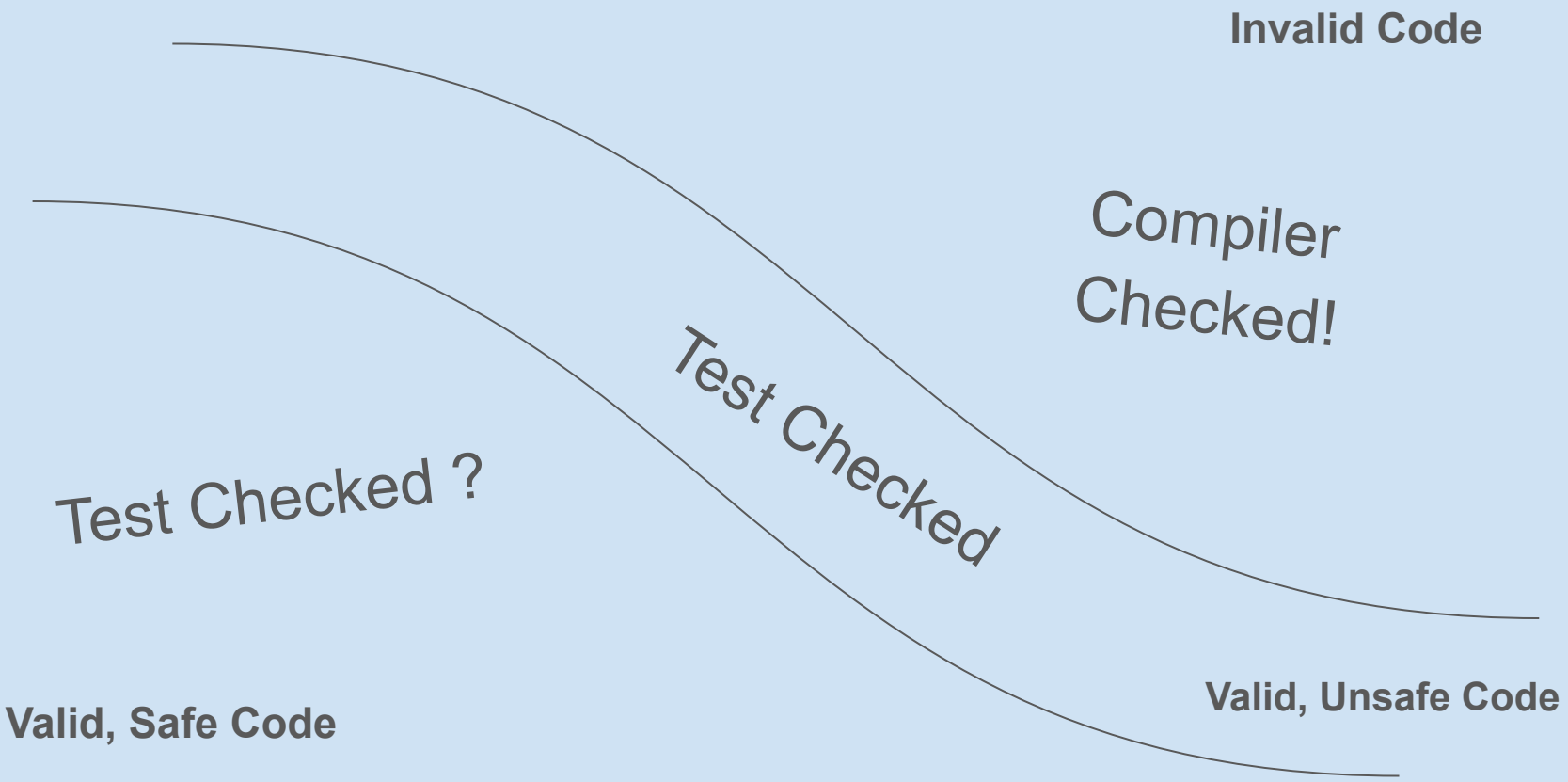
Testing provides weak guarantees for concurrent systems*, can we do better? How?

* Do not interpret this as “you shouldn’t write tests for concurrent programs”!!!!! Always test your code!!!

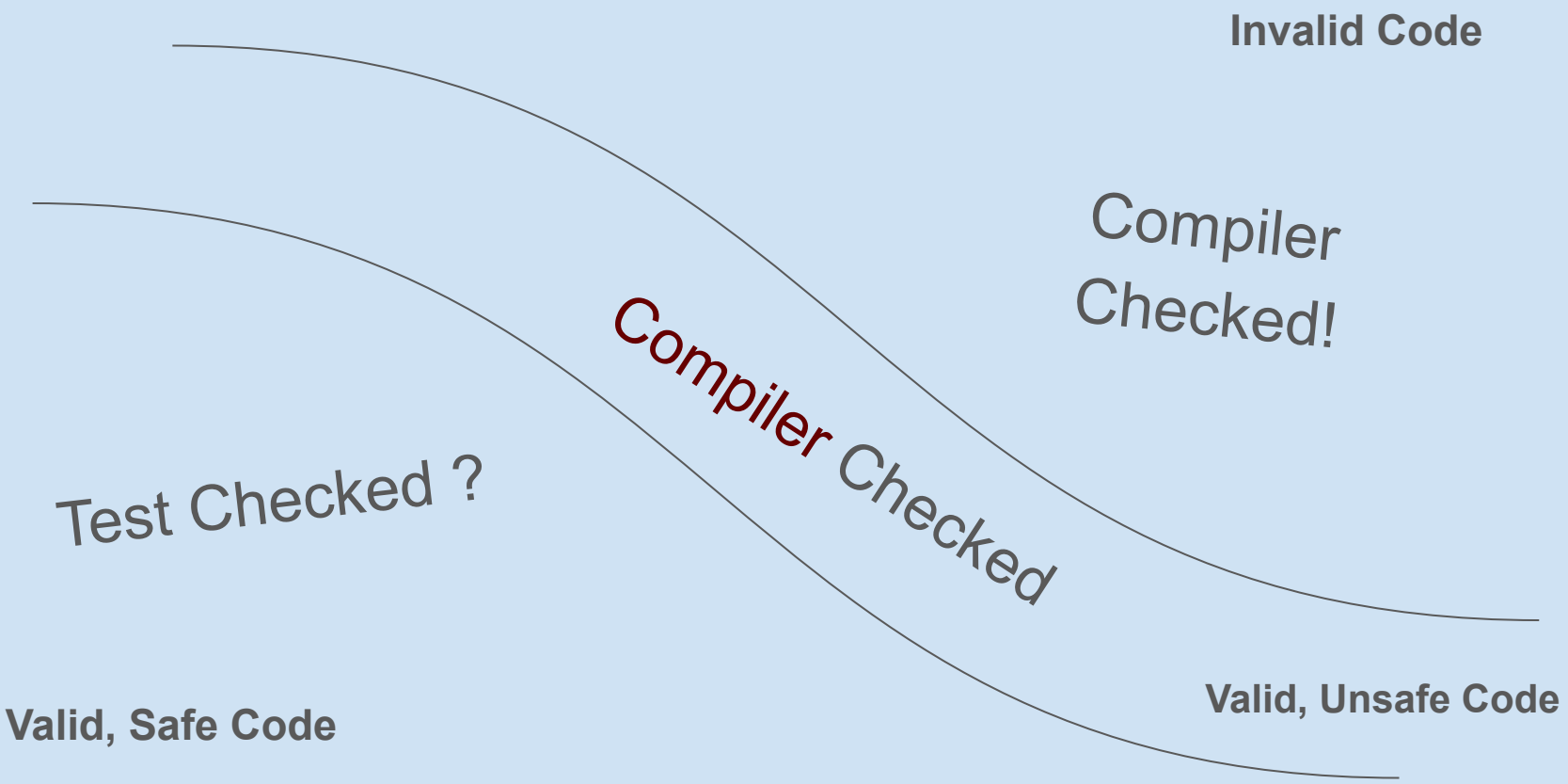
The Categories of Code



The Categories of Code



The Categories of Code



A Stronger System

If we impose more rules onto what code is and isn't "valid", the compiler becomes our proof of correctness tool

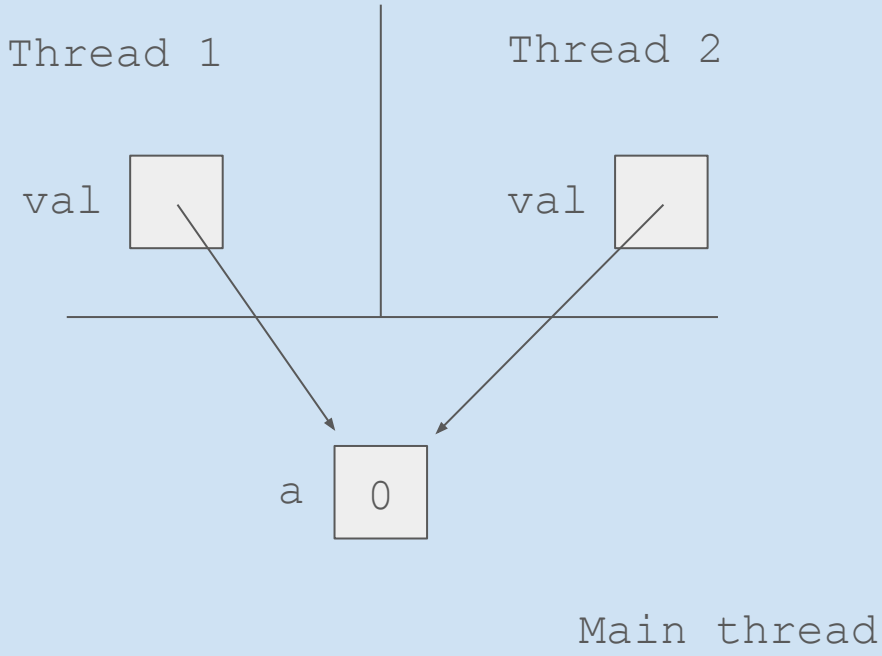
What rules do we pick? What problems do we want to solve?

A Stronger Concurrent System

Concurrency problems arise when data is accessed and modified at the same time.

```
fn incr(&val):
    val += 1

fn main():
    a = 0
    thread_run(incr, &a)
    thread_run(incr, &a)
    assert a == 2
```



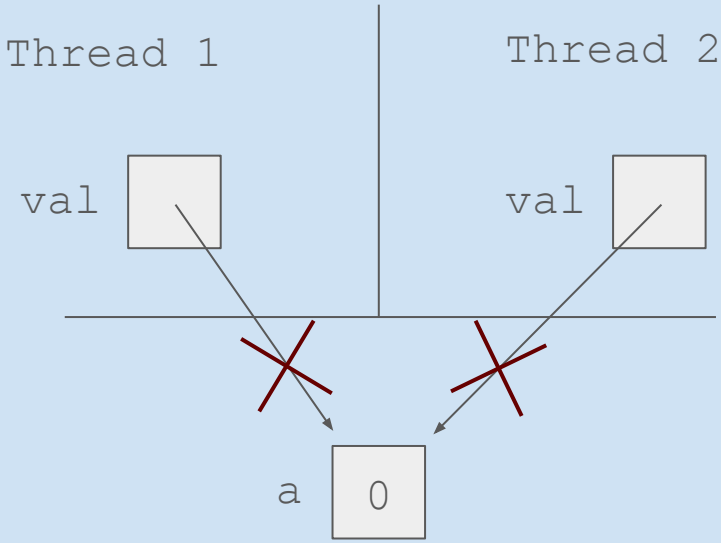
A Stronger Concurrent System

Concurrency problems arise when data is accessed and modified at the same time.

```
fn incr(&val):
    val += 1


fn main():
    a = 0
    thread_run(incr, &a)
    thread_run(incr, &a)
    assert a == 2
```

Compiler can detect this!



Disallow creating multiple ways of mutating a single piece of data!

Main thread

Data Race

2 instructions from different threads access the same memory

at least one is a write

no synchronization mandating an order between the accesses

A Stronger Concurrent System

- Multiple access is only a problem if you are planning to **write** data via a reference. Let's introduce "mutable references" (`&mut`) and "immutable references" (`&`).

```
fn incr(&mut val):
    val += 1
```

Note, we now need to tell the compiler to expect a reference it can write to

```
fn main():
    a = 0
    thread_run(incr, &mut a)
    thread_run(incr, &mut a)
    assert a == 2
```

Two mutable references → compiler error!

A Stronger Concurrent System

- Multiple access is only a problem if you are planning to **write** data via a reference. Let's introduce "mutable references" (**`&mut`**) and "immutable references" (**`&`**).

```
fn func(&val):
    print(val)


fn main():
    a = 0
    thread_run(func, &a)
    thread_run(func, &a)
    assert a == 2
```

Compiler knows we won't be writing to **`&val`**

Two "reader" references → this code is race safe → no compiler error!

A Stronger Concurrent System

- Multiple access is only a problem if you are planning to **write** data via a reference. Let's introduce "mutable references" (`&mut`) and "immutable references" (`&`).

```
fn incr(&mut val):
    return val + 1

fn main():
    a = 0
    thread_run(incr, &a)
    thread_run(incr, &a)
    assert a == 2
```

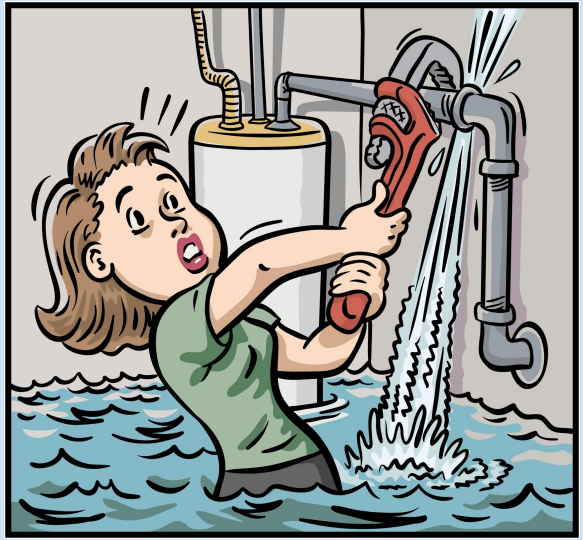
"Read only" references, but you are trying to write to a variable!

This is a type error!

A Stronger Memory System

Can the compiler help us manage memory?

- Java, Python, Go, C# are all garbage collected (GC) languages, GCs are slow.
- C / C++ require explicit freeing of memory, which is hard to do correctly at times.



A Stronger Memory System

Memory is safe to access when assigned to, and freed when we are done using it.

- Can we reason about when we can use certain memory?

```
fn main() {
    int* a = (int*) malloc(sizeof(int);
    print(*a);
    *a = 10;
    /* a bunch of operations */
    print(*a);
    free(a);
    print(*a);
}
```

Is this a safe use of `a`?

A Stronger Memory System

Memory is safe to access when assigned to, and freed when we are done using it.

- Can we reason about when we can use certain memory?

```
fn main() {
    int* a = (int*) malloc(sizeof(int);
    print(*a);
    *a = 10;
    /* a bunch of operations */
    print(*a);
    free(a);
    print(*a);
}
```

Is this a safe use of `a`?

No, it is yet to be initialized.

A Stronger Memory System

Memory is safe to access when assigned to, and freed when we are done using it.

- Can we reason about when we can use certain memory?

```
fn main() {
    int* a = (int*) malloc(sizeof(int);
    print(*a);
    *a = 10;
    /* a bunch of operations */
    print(*a);
    free(a);
    print(*a);
}
```

Is this a safe use of `a`?

A Stronger Memory System

Memory is safe to access when assigned to, and freed when we are done using it.

- Can we reason about when we can use certain memory?

```
fn main() {
    int* a = (int*) malloc(sizeof(int);
    print(*a);
    *a = 10;
    /* a bunch of operations */
    print(*a);
    free(a);
    print(*a);
}
```

Is this a safe use of `a`?

Yes!

A Stronger Memory System

Memory is safe to access when assigned to, and freed when we are done using it.

- Can we reason about when we can use certain memory?

```
fn main() {
    int* a = (int*) malloc(sizeof(int);
    print(*a);
    *a = 10;
    /* a bunch of operations */
    print(*a);
    free(a);
    print(*a);
}
```

Is this a safe use of `a`?

A Stronger Memory System

Memory is safe to access when assigned to, and freed when we are done using it.

- Can we reason about when we can use certain memory?

```
fn main() {
    int* a = (int*) malloc(sizeof(int);
    print(*a);
    *a = 10;
    /* a bunch of operations */
    print(*a);
    free(a);
    print(*a);
}
```

No!

Is this a safe use of `a`?

A Stronger Memory System

Memory is safe to access when assigned to, and freed when we are done using it.

- Can we reason about when we can use certain memory?

```
fn main() {
    int* a = (int*) malloc(sizeof(int);
    print(*a);
    *a = 10;
    /* a bunch of operations */
    print(*a);
    free(a);
    print(*a);
}
```

Idea: let's associate memory with a single *owner* variable, the owner is the instance that is responsible for freeing.

`a` owns the memory that stores 10.

A Stronger Memory System

Memory is safe to access when assigned to, and freed when we are done using it.

- Can we reason about when we can use certain memory?

```
fn main() {
    int* a = (int*) malloc(sizeof(int);
    print(*a);
    *a = 10;
    /* a bunch of operations */
    print(*a);
    free(a);
    print(*a);
}
```

Observe: When **a** leaves the scope, then there is no way of accessing it's associated memory, and therefore we can clean it up!

We don't need to free anymore, as the memory will be freed automatically at the end of main!

A Stronger Type System

Types are how programmers communicate with with the compiler.

- Binaries don’t have any type information in them, the processor just accesses memory via an address and an expected data offset / length

Strong types enforce invariants.

A Stronger Type System

If you create a reference to a variable, the compiler must be able to prove that the reference goes out of scope before the thing it points to does.

No casting! No undefined behavior!

Introducing Rust!

Rust is a modern systems programming language focusing on **safety**, **speed**, and **concurrency**. It accomplishes these goals by being memory safe without using garbage collection.

– Rust By Example

Rust programmers are called ‘Rustaceans’

Rust

- Created in 2006 by Graydon Hoare (no relation)
  - Sponsored by Mozilla in 2009
  - Multi-paradigm, general purpose programming language
  - Adopted by major companies and governance via Rust Foundation
  - Rust became the second ‘main’ language in Linux Kernel 6.1

- Characteristics
  - Aims to support efficient, fearless, concurrent systems programming
  - Memory safe with rich type system
  - Ergonomic developer experience
  - Interoperable with C/C++

Hello World in Rust

```rust
fn main() {
    println!("Hello, World!");
}
```

```rust
fn main() {
    let unit = "CSE";
    let course_num: u16 = 334;
    let term = String::from("Winter 2026");

    println!("Hello {} {}, {} edition", unit, course_num, term);
}
```

hello_cse334.rs

```
$ rustc hello_cse334.rs
$ ./hello_cse334
Hello CSE 334, Winter 2026 edition
```